E-Learning Platform using Binary serch Tree (BST)

P.BINDU LOVELY
DEPT. NAME:SCHOOL OF COMPUTING AND DATA SCIENCE
SAI UNIVERSITY
CHENNAI, INDIA

J.MUNI BHARGAV REDDY
DEPT. NAME:SCHOOL OF COMPUTING AND DATA SCIENCE
SAI UNIVERSITY
CHENNAI, INDIA

ABSTRACT

This project presents an efficient Course Management System implemented using Binary Search Trees (BST) in the C programming language. The system organizes course content hierarchically into modules and lessons, where both modules and lessons are maintained using BST structures for efficient storage, retrieval, and management. The application supports essential operations such as insertion, deletion, updating, searching, and display of modules and lessons in sorted order. By leveraging BST properties, the system achieves improved performance compared to linear data structures like linked lists. The implementation also ensures proper memory management to avoid leaks. This project demonstrates how tree-based data structures can be effectively used in real-world applications like e-learning platforms.

Keywords— Binary Search Tree, Course Management System, Data Structures, C Programming, E-learning Platform

I. INTRODUCTION

With the rapid growth of digital education, efficient management of course content has become essential. Traditional systems often rely on linear data structures, which can lead to inefficient search and update operations. This project introduces a Course Management System using Binary Search Trees (BST), enabling faster and more structured data handling.The system organizes courses into modules and lessons, maintaining them in sorted order. This approach improves performance and scalability, making it suitable for modern e-learning environments.

II. OBJECTIVES

The primary objective of this project is to design and implement an efficient Course Management System using a Binary Search Tree (BST) to enhance the organization and handling of course data within an e-learning platform. The system aims to support fundamental operations such as insertion, deletion, updating, and searching of course modules and lessons with improved speed and accuracy. It also focuses on maintaining course content in a sorted order to facilitate structured access, easy navigation, and better visualization for users. Additionally, the project emphasizes proper memory management to ensure optimal utilization of system resources. Another key objective is to develop a scalable and reliable solution capable of efficiently managing large volumes of educational content, thereby meeting the evolving demands of modern e-learning platforms.

III. SCOPE OF PROJECT

The system is designed for academic purposes to demonstrate the use of BST in real-world applications. It can be used by students and instructors to organize course material efficiently. The scope is limited to console-based interaction and does not include graphical interfaces or database integration.

IV. LITERATURE REVIEW

Binary Search Trees are widely used in computer science for efficient data organization. Compared to linear structures, BST offers faster searching and sorting capabilities. Previous studies have shown that hierarchical data structures improve performance in applications such as file systems, databases, and learning platforms. This project applies BST concepts to manage educational content effectively.

V. METHODOLOGY

The system is implemented using a hierarchical Binary Search Tree (BST) approach, where a course acts as the root node, modules are stored as BST nodes, and each module further contains a BST of lessons. This structure enables efficient organization and management of course content. Various operations are supported within the system to ensure smooth functionality. Insertion of nodes is performed based on alphabetical order using the strcmp() function, ensuring that the data remains sorted. Searching is carried out through recursive BST traversal, allowing quick access to specific modules or lessons. Deletion is handled carefully by considering all possible cases, including leaf nodes, nodes with a single child, and nodes with two children. The update operation allows dynamic replacement of node values when modifications are required. Finally, the display of course content is achieved through inorder traversal, which maintains and presents the data in a sorted and structured manner.

VI. TECHNOLOGY USED

Programming Language: C

Data Structure: Binary Search Tree (BST)

Compiler: GCC

Platform: (Windows/Linux/Mac)

Interface: Console faced

Input method: Standard Input(scanf)

Output method: Console Output(printf)

VII. ARCHITECTURE/SYSTEM DESIGN

The system follows a hierarchical tree-based design:

Course
→ Modules (BST)
→ Lessons (BST inside each module)

Each module node contains pointers to left and right child modules, and each lesson node similarly maintains left and right pointers. This structure ensures efficient organization and retrieval of data.

VIII. IMPLEMENTATION DETAILS

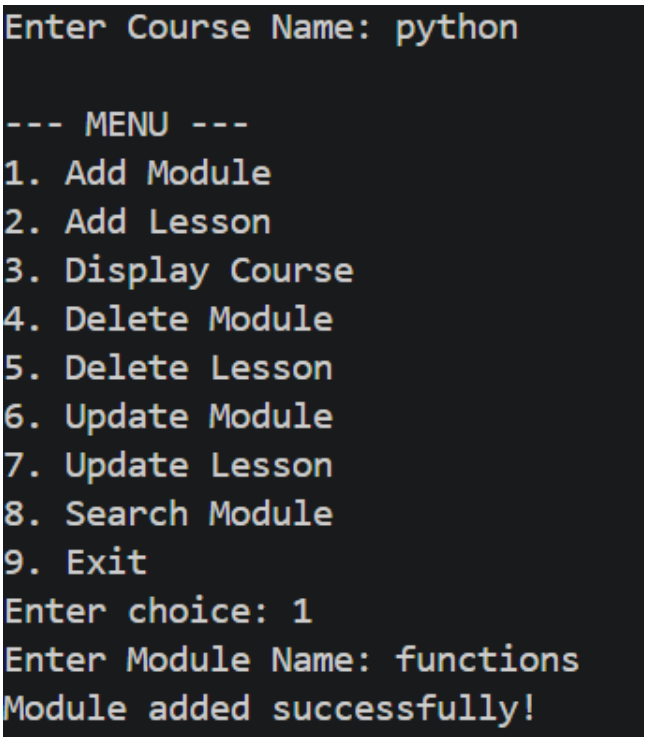
The program is implemented using structures for Course, Module, and Lesson. Dynamic memory allocation is used for storing strings and nodes. Key functions include:

Functions	***Time complexity***
1.insertModule()	O(log n)
2.insertLesson()	O(log n)
3.findModule()	O(log n)
4.findLesson()	O(log n)
5.deleteModuleNode()	O(log n)
6.deleteLessonNode()	O(log n)
7.updateModule()	O(log n)
8.updateLesson()	O(log n)
9.displayCourse()	O(n)
10.freeCourse()	O(n)

BST Operations**:** Efficient insertion, searching,deletion, and updating of modules and lessons using Binary Search Tree logic. Sorted Display**:** Uses inorder traversal to display course content in alphabetical (sorted) order. Memory Management**:** Proper allocation and deallocation of memory to avoid leaks.

IX. RESULTS / OUTPUT

Sample output:



X. CONCLUSION

The Course Management System using Binary Search Trees successfully demonstrates efficient data organization and retrieval. Compared to linear data structures, BST improves performance for search and update operations. The project is simple, scalable, and forms a strong foundation for advanced systems.

XI. FUTURE WORK

Future enhancements may include:

Graphical User Interface (GUI)

Database integration

Self-balancing trees (AVL / Red-Black Tree)

Web-based e-learning platform

User authentication system

XII. REFERENCES

[**1**] GeeksforGeeks, “Binary Search Tree Data Structure,” GeeksforGeeks, 2024. Available:https://www.geeksforgeeks.org/dsa/binary-search-tree-data-structure/

[**2**] [2] Programiz, “Binary Search Tree (BST),” 2023. Available: https://www.programiz.com/dsa/binary-search-tree

[**3**] Baeldung, “Binary Search Trees Guide,” Baeldung, 2022. Available: https://www.baeldung.com/cs/binary-search-trees